

Praca Dyplomowa Inżynierska

Adam Kliś
217584

Projekt systemu autoryzacji w oparciu o funkcjonalność tokenów

Design of an authorization system based on token functionality

Praca dyplomowa na kierunku:
Informatyka

Praca wykonana pod kierunkiem
dr inż. Artur Krupa
Katedra Informatyki / Instytut Informatyki Technicznej

Warszawa, rok 2026



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Słownik pojęć

- **Autoryzacja** – proces weryfikacji, czy dany (już uwierzytelniony) użytkownik posiada uprawnienia do wykonania określonej operacji.
- **Uwierzytelnianie (Authentication)** – proces weryfikacji tożsamości użytkownika (np. na podstawie loginu i hasła).
- **Token** – cyfrowy nośnik informacji, często zabezpieczony kryptograficznie, reprezentujący tożsamość lub uprawnienia użytkownika.
- **API (Application Programming Interface)** – interfejs programistyczny aplikacji pozwalający na komunikację między systemami.
- **Zero Trust** – model bezpieczeństwa zakładający, że żadne urządzenie ani użytkownik nie jest domyślnie zaufany, nawet jeśli znajduje się wewnątrz sieci korporacyjnej.

Spis treści

Słownik pojęć	3
1 Wstęp	5
2 Przegląd technologii i stan wiedzy	6
2.1 Uwierzytelnianie a autoryzacja	6
2.2 Protokół HTTP i problem braku stanu	6
2.3 Ewolucja: Od sesji stanowych do bezstanowych tokenów	6
2.4 Anatomia standardu JSON Web Token (JWT)	9
2.5 Paradygmat Inversion of Control i mechanizm DI	9
2.6 Środowisko uruchomieniowe i persystencja danych	10
3 Projekt i architektura systemu	11
3.1 Koncepcja autorskiego tokenu	11
3.2 Algorytm podpisu cyfrowego	11
3.3 Weryfikacja tokenu i integralność	12
3.4 Zarządzanie rolami przy użyciu masek bitowych	13
3.5 Analiza złożoności obliczeniowej i pamięciowej	13
3.6 Cykl życia sesji i rotacja tokenów odświeżających	14
3.7 Diagram klas: Warstwa danych	14
3.8 Diagram klas: Kontrakty i modele	15
3.9 Diagram klas: Logika i punkty dostępu	15
3.10 Diagram przypadków użycia	16
3.11 Diagram sekwencji weryfikacji uprawnień	16
4 Implementacja rozwiązania	18
4.1 Logika biznesowa i potok ASP.NET Core	18
4.2 Rejestracja i bezpieczne przechowywanie haseł	18
4.3 Proces logowania i weryfikacji tożsamości	19
4.4 Implementacja odświeżania sesji i rotacji tokenów	20
4.5 Konfiguracja i dynamiczne rozszerzanie ładunku (Custom Payload)	21
4.6 Automatyzacja zarządzania i migracja ról (Bootstrapper)	23
4.6.1 Dynamiczne skanowanie atrybutów	24
4.6.2 Proces bezpiecznej migracji uprawnień	24
4.7 Implementacja operacji kryptograficznych i ochrona przed atakami	25
4.8 Filtry potoku HTTP i weryfikacja tożsamości	27
4.9 Zaawansowana autoryzacja oparta o role	28
5 Analiza bezpieczeństwa i testowanie	31
5.1 Zgodność ze standardem OWASP Top 10	31
5.2 Automatyzacja testów bezpieczeństwa kryptograficznego	32
5.3 Analiza wydajnościowa i narzut obliczeniowy	33
5.4 Ograniczenia systemu i obszary dalszego rozwoju	34
6 Podsumowanie	35

1 Wstęp

Wraz z dynamicznym rozwojem aplikacji internetowych oraz popularyzacją architektury rozproszonej, mechanizmy uwierzytelniania i autoryzacji stały się jednym z najbardziej krytycznych elementów systemów informatycznych. Zapewnienie bezpieczeństwa danych, przy jednoczesnym zachowaniu wysokiej wydajności i skalowalności, stanowi wyzwanie, z którym inżynierowie oprogramowania mierzą się od początków istnienia sieci WWW. We wcześniejszych fazach rozwoju aplikacji internetowych serwery generowały statyczne strony HTML, a potrzeba śledzenia tożsamości użytkownika była marginalna. Z czasem standardem stało się podejście oparte na sesjach, gdzie serwer tworzył w pamięci operacyjnej obiekt sesji, a do przeglądarki klienta wysyłał jedynie identyfikator zapisywany w pliku cookie. Choć bezpieczne, rozwiązanie to utrudniało skalowanie poziome w nowoczesnych architekturach mikroserwisowych.

Współczesne podejście do cyberbezpieczeństwa coraz częściej opiera się na paradygmacie *Zero Trust* (architekturze zerowego zaufania). Koncepcja ta zrywa z tradycyjnym modelem "zaufanej sieci wewnętrznej". W modelu *Zero Trust* zakłada się, że zagrożenie może pojawić się zewsząd, dlatego każde, nawet wewnętrzne żądanie dostępu do zasobów musi zostać rygorystycznie zweryfikowane. Oznacza to, że uwierzytelnienie przy wejściu do systemu nie jest wystarczające – niezbędna jest ciągła weryfikacja uprawnień (autoryzacja) przy każdym pojedynczym zapytaniu. Aby realizować ten postulat wydajnie, branża oprogramowania odeszła od obciążających serwery sesji na rzecz lekkich, kryptograficznie podpisanych poświadczeń.

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz implementacja autorskiego, wysoce zoptymalizowanego systemu autoryzacji w środowisku .NET, opartego o funkcjonalność tokenów. Stworzone rozwiązanie stanowi alternatywę dla standardu JWT, optymalizując narzut sieciowy oraz proces walidacji ról za pomocą operacji bitowych. Praca została podzielona na logiczne etapy, prowadząc czytelnika od przeglądu teorii i stosu technologicznego (Rozdział 2), poprzez projekt architektury i diagramy pozbawione szczegółów implementacyjnych (Rozdział 3), aż po konkretną realizację programistyczną (Rozdział 4). Dopełnieniem procesu inżynierskiego jest testowanie rozwiązania i analiza jego bezpieczeństwa w kontekście standardów branżowych, takich jak OWASP Top 10 (Rozdział 5), co ostatecznie prowadzi do podsumowania uzyskanych wyników.

2 Przegląd technologii i stan wiedzy

Aby w pełni zrozumieć architekturę proponowanego rozwiązania, konieczne jest omówienie fundamentów teoretycznych oraz technologii wykorzystywanych w nowoczesnych aplikacjach webowych. Rozdział ten wprowadza pojęcia protokołu HTTP, dogłębnie analizuje ewolucję mechanizmów utrzymania stanu, poddaje krytyce popularny standard JWT oraz definiuje stos technologiczny wybrany do implementacji autorskiego systemu.

2.1 Uwierzytelnianie a autoryzacja

W inżynierii bezpieczeństwa oprogramowania pojęcia uwierzytelniania i autoryzacji są często używane zamiennie, co stanowi poważny błąd terminologiczny. Zrozumienie różnicy między nimi jest kluczowe dla poprawnego modelowania systemów dostępu.

Uwierzytelnianie (ang. Authentication - AuthN) to proces weryfikacji tożsamości użytkownika lub systemu. Odpowiada na pytanie: "Kim jesteś?". W tradycyjnych systemach proces ten realizowany jest najczęściej poprzez podanie loginu i hasła. W rozwiązaniach bardziej zaawansowanych stosuje się biometrię lub uwierzytelnianie wieloskładnikowe (MFA).

Autoryzacja (ang. Authorization - AuthZ) to proces decyzyjny następujący zawsze po udanym uwierzytelnieniu. Odpowiada na pytanie: "Czy masz prawo to zrobić?". Nawet jeśli tożsamość użytkownika została poprawnie zweryfikowana, system autoryzacyjny musi sprawdzić, czy jego rola uprawnia go do dostępu do konkretnego zasobu (np. usunięcia rekordu z bazy danych). Niniejsza praca skupia się na optymalizacji obu tych procesów, ze szczególnym naciskiem na minimalizację kosztów obliczeniowych podczas ciągłej autoryzacji zapytań sieciowych.

2.2 Protokół HTTP i problem braku stanu

Protokół HTTP (ang. *Hypertext Transfer Protocol*), będący fundamentem komunikacji w sieci Web, z definicji jest protokołem bezstanowym (ang. *Stateless*). Oznacza to, że każde żądanie wysyłane przez klienta do serwera traktowane jest jako całkowicie niezależna operacja. Serwer na poziomie sieciowym nie przechowuje żadnych informacji o poprzednich interakcjach z danym klientem.

Z inżynierskiego punktu widzenia bezstanowość HTTP jest jego ogromną zaletą. Pozwala ona na szybkie obsługiwanie tysięcy jednoczesnych połączeń bez drastycznego przyrostu zużycia pamięci operacyjnej (RAM) serwera. Wymusza to jednak na twórcach oprogramowania aplikacyjnego budowę mechanizmów, które do każdego żądania dołączają dowód tożsamości użytkownika.

2.3 Ewolucja: Od sesji stanowych do bezstanowych tokenów

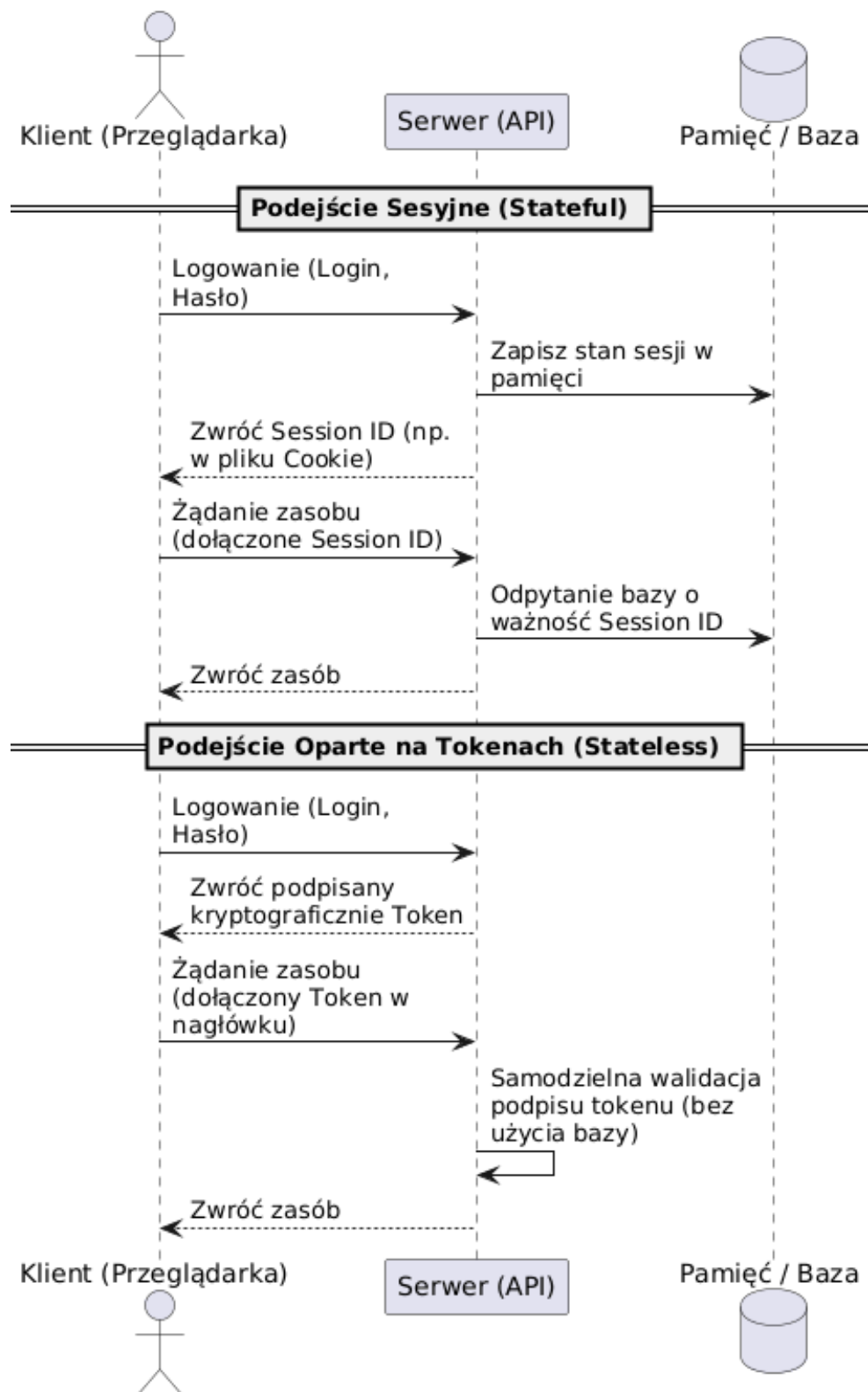
Początkowo standardem utrzymywania ciągłości pracy użytkownika było podejście oparte na sesjach (ang. *Session-based Authentication*). W tym modelu serwer, po zweryfikowaniu hasła, alokował w swojej pamięci operacyjnej obiekt sesji, a klientowi odsyłał jedynie krótki identyfikator (tzw. *Session ID*) za pomocą mechanizmu ciasteczek (ang. *Cookies*). Przeglądarka automatycznie dołączała ten identyfikator do kolejnych zapytań, a serwer za każdym

razem musiał przeszukiwać swoją pamięć lub bazę danych, aby upewnić się, że sesja jest nadal ważna.

Złotą erę tego podejścia zakończył rozwój architektury rozproszonej i chmury obliczeniowej. W środowiskach mikroserwisowych, gdzie ruch rozkładany jest na wiele niezależnych instancji serwerowych (skalowanie poziome), stanowe sesje stały się tzw. wąskim gardłem (ang. *bottleneck*). Jeśli pierwsze żądanie użytkownika trafiło na serwer A, który zapisał sesję, a kolejne na serwer B, uwierzytelnienie kończyło się błędem.

Rozwiązaniem tego problemu stały się tokeny. W modelu *Token-based Authentication* serwer podpisuje pakiet informacji reprezentujący tożsamość użytkownika, a następnie odsyła go do klienta. Klient przechowuje token i dołącza go do kolejnych żądań HTTP (najczęściej w nagłówku *Authorization*). Różnice w przepływie sterowania obu mechanizmów zobrazowano na Rysunku 2.1.

Jak widać na schemacie, podejście oparte na tokenach całkowicie odciąża bazę danych z procesu ciągłej autoryzacji – serwer samodzielnie weryfikuje podpis matematyczny.



Rysunek 2.1. Porównanie architektury stanowej (sesyjnej) z bezstanową (tokenową)

2.4 Anatomia standardu JSON Web Token (JWT)

Najpopularniejszym rynkowym formatem tokenu jest standard JWT (zdefiniowany w dokumencie RFC 7519). Klasyczny token JWT to długi ciąg znaków (często przekraczający 500 bajtów), który składa się z trzech części oddzielonych kropkami:

1. Nagłówek (Header): Definiuje typ tokenu oraz algorytm kryptograficzny użyty do podpisu. Zazwyczaj ma postać prostego obiektu JSON:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Ładunek (Payload): Miejsce przechowywania oświadczeń (ang. *claims*), czyli faktycznych danych o użytkowniku, takich jak jego identyfikator (**sub**), data wygaśnięcia (**exp**) czy posiadane uprawnienia. W standardzie JWT role przesyłane są zazwyczaj jako pełne ciągi tekstowe:

```
{
  "sub": "1234567890",
  "name": "Jan Kowalski",
  "roles": ["admin", "moderator", "user"],
  "exp": 1516239022
}
```

3. Podpis (Signature): Kryptograficzny skrót obliczony z zakodowanego nagłówka i ładunku przy użyciu klucza tajnego.

Krytyczna analiza tego standardu uwidacznia jego redundancję w zamkniętych środowiskach mikroserwisowych. Przesyłanie w każdym pojedynczym żądaniu nagłówka informującego o typie algorytmu jest zbędnym narzutem danych, ponieważ aplikacja odczytująca token posiada już tę wiedzę z własnej konfiguracji wewnętrznej. Ponadto, tekstowy zapis ról potęguje objętość ładunku. Wyeliminowanie tych nadmiarowych elementów stało się główną motywacją do stworzenia autorskiego formatu opisanego w niniejszej pracy.

2.5 Paradygmat Inversion of Control i mechanizm DI

Zastosowany stos technologiczny wymusza na programistach przestrzeganie dobrych praktyk projektowych. Jednym z najważniejszych paradygmatów zastosowanych w architekturze biblioteki jest odwrócenie sterowania (ang. *Inversion of Control* – IoC) realizowane poprzez wzorzec Wstrzykiwania Zależności (ang. *Dependency Injection* – DI).

Zasada ta jest bezpośrednio powiązana z literą "D" w akronimie SOLID (ang. *Dependency Inversion Principle*), która mówi, że moduły wysokopoziomowe nie powinny zależeć od modułów niskopoziomowych, a obie warstwy powinny opierać się na abstrakcjach. Zamiast bezpośrednio tworzyć instancje obiektów (np. połączeń do bazy danych) wewnątrz klas przy użyciu słowa kluczowego **new**, komponenty systemu deklarują swoje zależności poprzez interfejsy w konstruktorach.

W ekosystemie ASP.NET Core wbudowany kontener DI samodzielnie zarządza cyklem życia obiektów. Rozwiązanie to jest krytyczne dla autorskiego systemu autoryzacji, ponieważ zapewnia mu wymaganą hermetyzację, wysoką testowalność (łatwe wstrzykiwanie atrap – tzw. mocków) oraz modułowość pozwalającą na integrację z dowolnym środowiskiem sieciowym.

2.6 Środowisko uruchomieniowe i persystencja danych

Do realizacji projektu wybrano środowisko uruchomieniowe **.NET** oraz język **C#**. Platforma ta, rozwijana przez firmę Microsoft, stanowi obecnie jeden z najbardziej zaawansowanych i wydajnych ekosystemów do budowy wielowątkowych aplikacji serwerowych (Web API).

Istotnym elementem architektury jest warstwa persystencji danych. Interakcja z relacyjną bazą danych odbywa się za pośrednictwem technologii Entity Framework Core (EF Core). Jest to zaawansowane narzędzie klasy ORM (ang. *Object-Relational Mapping*), które rozwiązuje problem tzw. niedopasowania impedancji (ang. *impedance mismatch*) pomiędzy światem programowania obiektowego a tabelami bazy danych.

Wykorzystanie EF Core pozwala na operowanie na bazie bez konieczności pisania jawnych zapytań w języku SQL. Kodowanie odbywa się przy pomocy składni LINQ (ang. *Language Integrated Query*), a framework automatycznie tłumaczy te komendy na bezpieczny i zoptymalizowany pod dany silnik dialekt SQL. Użycie tego narzędzia automatycznie zabezpiecza autorski system autoryzacji przed jednym z najgroźniejszych ataków sieciowych – *SQL Injection* – ponieważ wszystkie wartości wstawiane do bazy są domyślnie traktowane jako parametryzowane dane wejściowe.

3 Projekt i architektura systemu

Architektura omawianego systemu została zoptymalizowana pod kątem redukcji narzutu sieciowego oraz błyskawicznej walidacji. W tym rozdziale zaprezentowano projekt mechanizmów wewnętrznych, sformalizowane modele matematyczne algorytmów oraz przepływy procesów. Zgodnie z dobrymi praktykami dokumentacji architektonicznej, celowo pominięto na tym etapie bezpośrednie implementacje kodowe, skupiając się na logice systemowej.

3.1 Koncepcja autorskiego tokenu

W przeciwieństwie do powszechnych standardów, autorski token został zaprojektowany z myślą o minimalizacji przesyłanych bajtów. Jest to ciąg znaków składający się zaledwie z dwóch segmentów: ładunku (B) zawierającego właściwe oświadczenia i dane użytkownika oraz sumy kontrolnej (S).

Proces powstawania tokenu T można sformalizować za pomocą następującego równania:

$$T = \text{Base64Url}(B) + "." + \text{Base64Url}(S)$$

gdzie:

- T – finalny ciąg znaków reprezentujący token autoryzacyjny, przesyłany w nagłówku zapytania HTTP.
- B – ładunek (ang. *Body/Payload*), będący tekstową reprezentacją zserializowanego obiektu JSON, przechowującego dane sesji (np. zdekodowaną maskę ról oraz czas wygaśnięcia).
- S – suma kontrolna (ang. *Signature*), będąca binarnym wynikiem operacji kryptograficznej zabezpieczającej ładunek.
- $\text{Base64Url}()$ – funkcja kodująca dane do bezpiecznego formatu tekstowego.

Kluczowym zabiegiem inżynierskim w tym równaniu jest użycie algorytmu **Base64Url** zamiast standardowego Base64. Standardowe kodowanie wprowadza znaki takie jak $+$ oraz $/$, które posiadają specjalne znaczenie w adresach URL i nagłówkach HTTP (mogą prowadzić do błędów parsowania). Base64Url zastępuje je odpowiednio znakami $-$ (myślnik) oraz $_$ (podkreślenie), a także usuwa znaki dopełnienia $=$, co gwarantuje pełną kompatybilność protokołu sieciowego.

3.2 Algorytm podpisu cyfrowego

Bezpieczeństwo systemu opiera się na zapewnieniu integralności i niezaprzeczalności emitowanych poświadczeń. Do generowania sumy kontrolnej (S) wykorzystano algorytm symetryczny HMAC (ang. *Keyed-Hash Message Authentication Code*) w oparciu o silną funkcję skrótu SHA-256.

Wybór kryptografii symetrycznej podyktowany jest niespotykaną wydajnością oraz specyfiką aplikacji monolitycznych, gdzie serwer wydający i weryfikujący to ten sam podmiot, posiadający bezpieczny dostęp do pamięci operacyjnej. Pełny proces generowania podpisu σ można wyrazić wzorem matematycznym:

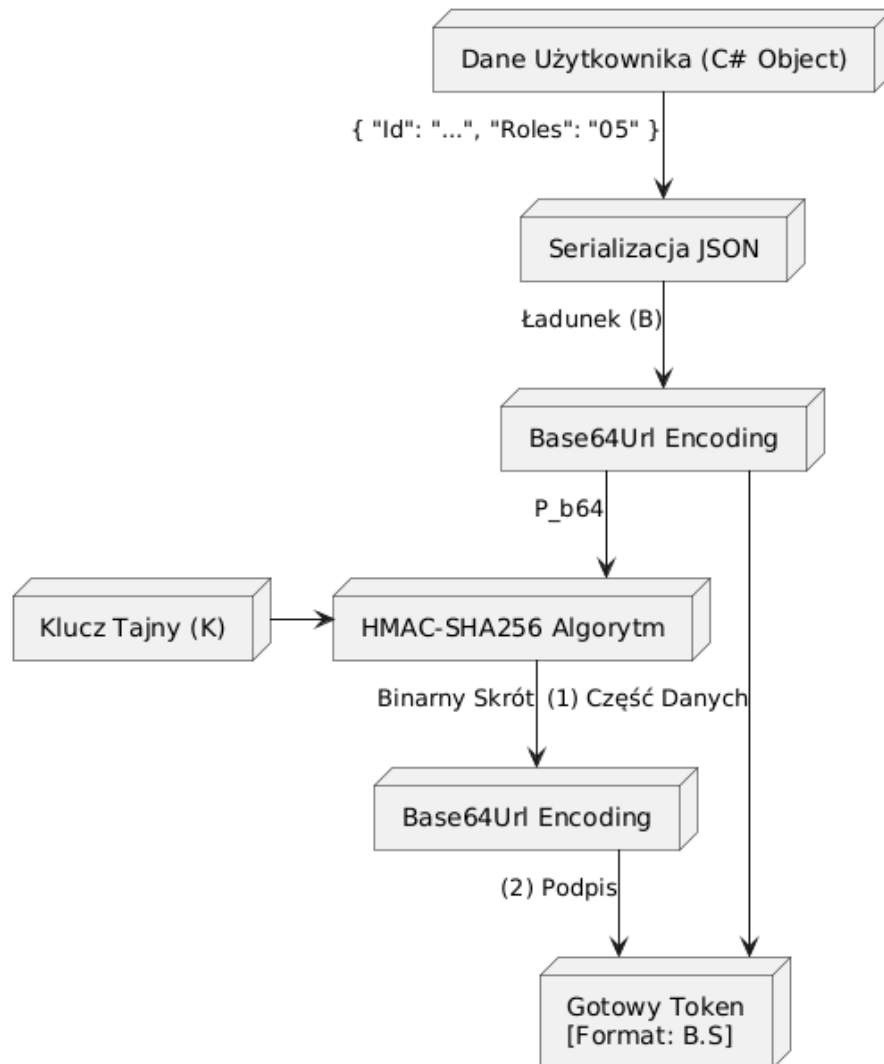
$$\sigma = \text{Base64Url}(\text{HMAC}_{\text{SHA256}}(K, P))$$

gdzie poszczególne zmienne oznaczają:

- σ – ostateczny, tekstowy podpis dołączany na końcu tokenu.

- K – klucz tajny (ang. *Secret Key*), będący ciągiem znaków o wysokiej entropii, zdefiniowanym w bezpiecznej konfiguracji środowiskowej serwera. Klucz ten nigdy nie jest przesyłany do klienta.
- P – ładunek wejściowy, czyli zakodowana już w Base64Url część danych ($P = \text{Base64Url}(B)$).
- $\text{HMAC}_{\text{SHA256}}$ – kryptograficzna funkcja mieszająca, która w bezpieczny sposób łączy klucz tajny z wiadomością, odporna na ataki typu *Length Extension Attack*.

Przepływ danych w procesie generowania tokenu, od momentu pobrania obiektu po uzyskanie końcowego ciągu znaków, zaprezentowano na Rysunku 3.1.



Rysunek 3.1. Architektura kryptograficzna generowania tokenu

3.3 Weryfikacja tokenu i integralność

Zrozumienie procesu weryfikacji jest kluczowe z perspektywy bezpieczeństwa. Weryfikacja nie polega na kryptograficznym "odszyfrowywaniu" sumy kontrolnej, lecz na procesie jej reprodukcji.

Kiedy serwer odbiera token z zapytania HTTP, rozdziela go używając znaku kropki jako separatora. Następnie, korzystając z przechwyconego ładunku P oraz własnego klucza symetrycznego K , ponownie wykonuje funkcję $\text{HMAC}_{\text{SHA256}}$. Jeśli nowo wyliczona sygnatura σ' jest matematycznie tożsama z sygnaturą σ nadesłaną przez klienta ($\sigma' == \sigma$), serwer

nabiera pewności, że dane nie uległy manipulacji. Ewentualna zmiana choćby jednego bitu w ładunku (np. próba zmiany identyfikatora roli) spowoduje drastyczną zmianę wyliczonego skrótu zjawiskiem lawinowym (ang. *Avalanche Effect*), co skutkuje natychmiastowym odrzuceniem żądania HTTP z kodem 401 Unauthorized.

3.4 Zarządzanie rolami przy użyciu masek bitowych

Największą innowacją zaprojektowanego systemu jest całkowite odejście od tekstowej reprezentacji uprawnień. Model ten opiera się na matematycznej teorii zbiorów oraz systemie wag dwójkowych.

Niech R stanowi zbiór wszystkich dostępnych ról w systemie, a każda rola $r \in R$ posiada unikalny indeks liczbowy $ID_r \in \{0, 1, 2, \dots, n\}$. Zbiór ról przypisanych do konkretnego użytkownika oznaczmy jako $U \subseteq R$. Maska bitowa M dla danego użytkownika obliczana jest jako suma potęg liczby 2:

$$M = \sum_{r \in U} 2^{ID_r}$$

Dla przykładu, jeżeli system posiada role "user" ($ID = 0$) oraz "admin" ($ID = 2$), a użytkownik posiada obie z nich, jego maska wynosi:

$$M = 2^0 + 2^2 = 1 + 4 = 5$$

Liczba dziesiętna 5 jest następnie konwertowana do formatu szesnastkowego (05) i w takiej skompresowanej formie umieszczana w ładunku tokenu.

Kiedy użytkownik żąda dostępu do zasobu zabezpieczonego konkretną rolą r_{req} , serwer weryfikujący nie musi przeszukiwać list ani struktur tekstowych. Wykonuje on wyłącznie jedną, natywną dla procesora operację iloczynu bitowego (AND). Dostęp zostaje przyznany wtedy i tylko wtedy, gdy spełniony jest warunek:

$$(M \text{ AND } 2^{ID_{req}}) \neq 0$$

Działanie to omija kosztowną alokację pamięci, czyniąc proces weryfikacji niezwykle wydajnym.

3.5 Analiza złożoności obliczeniowej i pamięciowej

Sformalizowanie ról do postaci masek bitowych przynosi wymierne korzyści, co można udowodnić za pomocą notacji asymptotycznej (*Big O Notation*).

W standardowym podejściu (klasyczny JWT), role weryfikowane są poprzez iteracyjne przeszukiwanie tablicy ciągów znaków. Dla N ról posiadanych przez użytkownika oraz M ról wymaganych przez punkt końcowy, złożoność obliczeniowa najgorszego przypadku wynosi $O(N \cdot M)$. Ponadto, operacje na łańcuchach znaków (ang. *string comparisons*) dodatkowo obciążają pamięć sterty (ang. *Heap*) oraz wirtualną maszynę odśmiecającą pamięć (*Garbage Collector*).

W mechanizmie autorskim, po początkowym zdekodowaniu krótkiej maski szesnastkowej, proces walidacji sprowadza się do wywołania operatorów sprzętowych procesora. Złożoność obliczeniowa porównania bitowego, bez względu na wielkość słownika ról w systemie, jest stała i wynosi zawsze $O(1)$. Złożoność pamięciowa została zredukowana do alokacji pojedynczych zmiennych typów prostych (ang. *Value Types*).

3.6 Cykl życia sesji i rotacja tokenów odświeżających

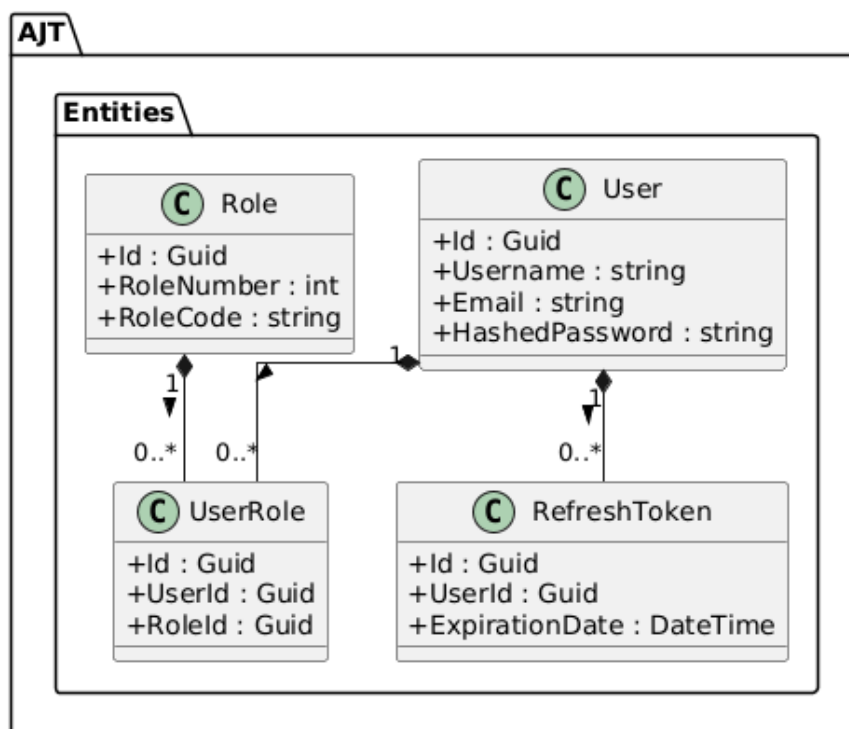
Krótki czas życia tokenu dostępowego (zazwyczaj od kilku do kilkunastu minut) stanowi fundament bezpieczeństwa w modelu bezstanowym. Ograniczenie to drastycznie minimalizuje okno czasowe, w którym przechwycony token mógłby posłużyć do wyrządzenia szkód w systemie.

Aby zapewnić ciągłość pracy bez wymuszania na użytkowniku ciągłego wpisywania hasła, wprowadzono długoterminowe tokeny odświeżające (ang. *Refresh Tokens*). Architektura zakłada wdrożenie mechanizmu rotacji (ang. *Refresh Token Rotation*).

Gdy token dostępowy traci ważność, aplikacja kliencka wysyła na dedykowany punkt końcowy token odświeżający. Jest to jedyny etap, w którym system wykonuje operację stanową (odpytuje relacyjną bazę danych). Po zweryfikowaniu kryptograficznym serwer upewnia się, że token istnieje w bazie i nie uległ przedawnieniu. Jeśli weryfikacja powiedzie się, stary token jest bezzwłocznie usuwany, a system emituje nową parę kluczy (nowy Access Token oraz nowy Refresh Token). Mechanizm ten gwarantuje, że skradziony token odświeżający będzie dla atakującego jednorazowy – gdy tylko prawowity użytkownik wygeneruje nową sesję, stary identyfikator zostanie bezpowrotnie unieważniony.

3.7 Diagram klas: Warstwa danych

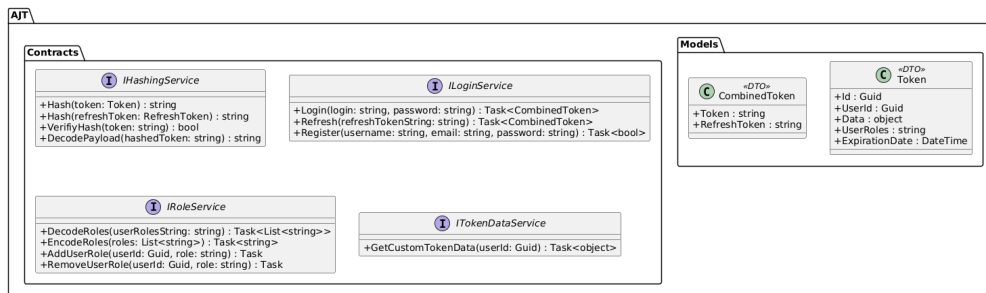
Z uwagi na zastosowanie wzorców inżynierskich system podzielono na logiczne pakiety, co zilustrowano za pomocą graficznej notacji UML. Rysunek 3.2 prezentuje encje bazodanowe mapujące obiekty C# na struktury relacyjne bazy. Widoczne jest powiązanie użytkownika z odpowiednimi tokenami odświeżającymi (relacja jeden-do-wielu), oraz przypisanie uprawnień poprzez tabelę asocjacyjną *UserRole*.



Rysunek 3.2. Diagram klas warstwy danych (Entities)

3.8 Diagram klas: Kontrakty i modele

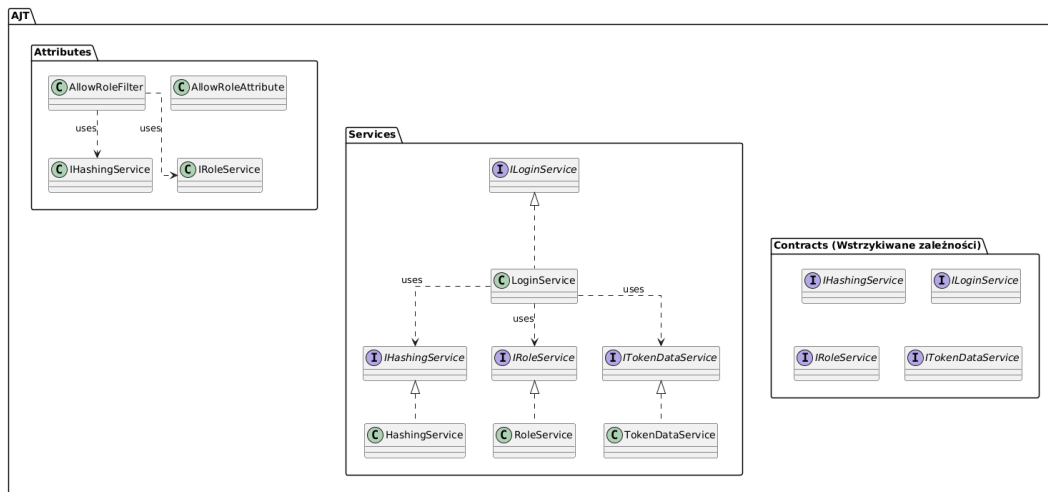
Rysunek 3.3 obrazuje obiekty transferu danych (DTO), takie jak model **Token** oraz jego zagnieźdzone elementy. Separacja abstrakcyjnych interfejsów (np. **IHashingService**) od ich implementacji fizycznych jest strukturalnym wymogiem obsługi wbudowanego kontenera Wstrzykiwania Zależności.



Rysunek 3.3. Diagram klas warstwy kontraktów i modeli

3.9 Diagram klas: Logika i punkty dostępu

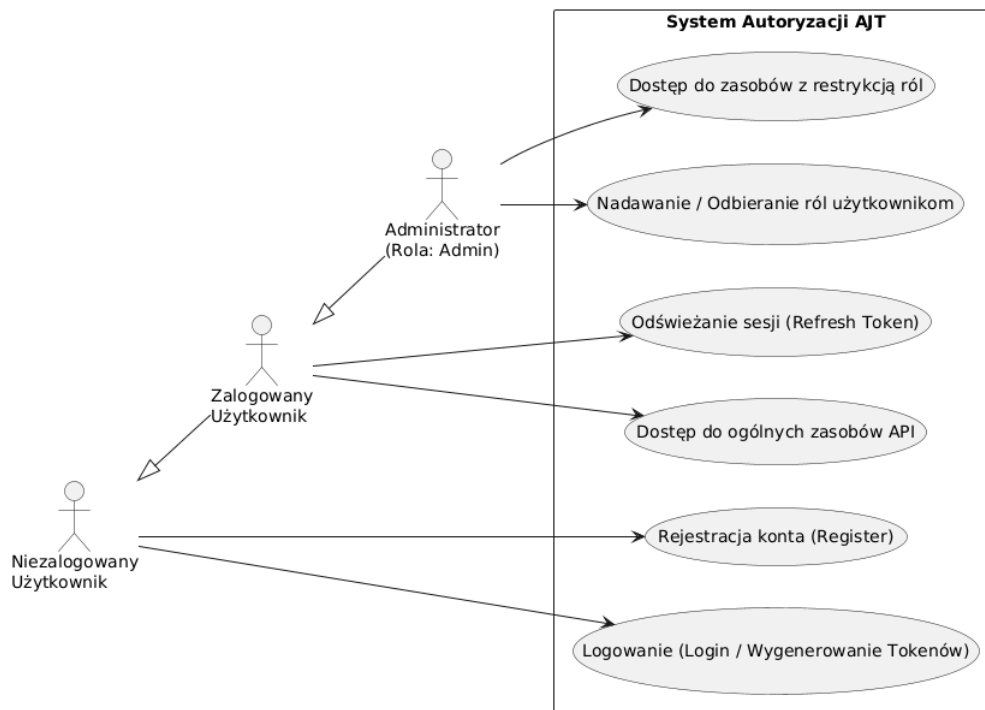
Ostatni schemat statyczny (Rysunek 3.4) skupia się na relacjach operacyjnych w warstwie biznesowej. Obrazuje zależność między wbudowanymi filtrami autoryzacyjnymi przechwytyjącymi żądania HTTP, a wstrzykiwanymi do nich implementacjami serwisów przetwarzającymi maski bitowe.



Rysunek 3.4. Diagram klas implementacji logiki biznesowej

3.10 Diagram przypadków użycia

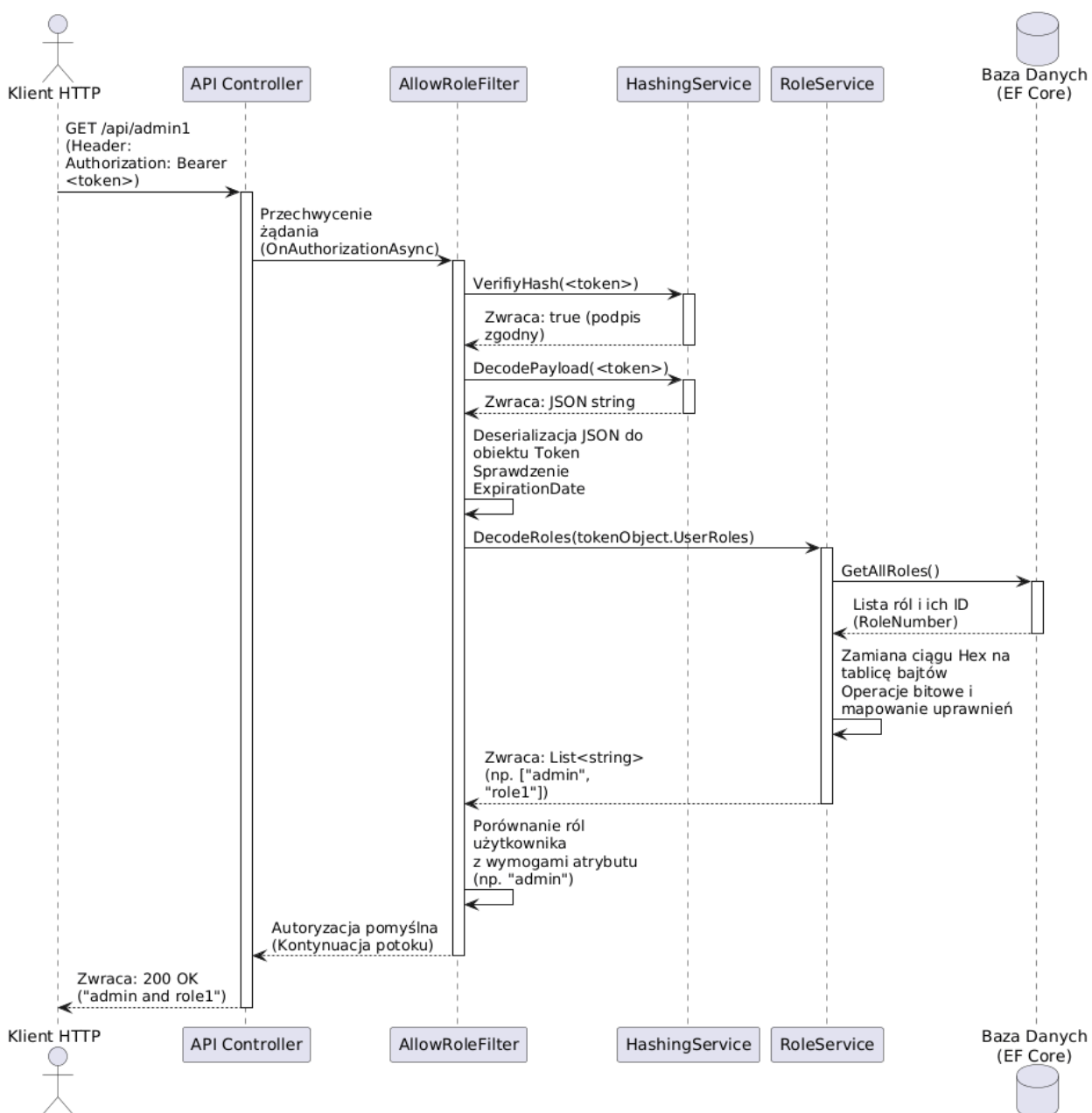
Zidentyfikowano trzech głównych aktorów wchodzących w interakcję z modulem (gość, użytkownik zalogowany, administrator). Jak zobrazowano na Rysunku 3.5, architektura opiera się na dziedziczeniu uprawnień, gdzie wyższy poziom dostępu wchłania kompetencje poziomu niższego, odblokowując zarazem administracyjne punkty dostępowe.



Rysunek 3.5. Diagram przypadków użycia systemu

3.11 Diagram sekwencji weryfikacji uprawnień

Sekwencyjny przepływ weryfikacji dostępu przedstawiono na Rysunku 3.6. Proces ten aktywuje się w momencie dotarcia żądania HTTP do serwera, tuż przed wywołaniem właściwej logiki domenowej. Przedstawiono w nim cykl życia walidacji: od weryfikacji kryptograficznej, przez dekodowanie matematycznej maski ról, aż po ostateczne przerwanie lub kontynuację potoku wykonawczego.



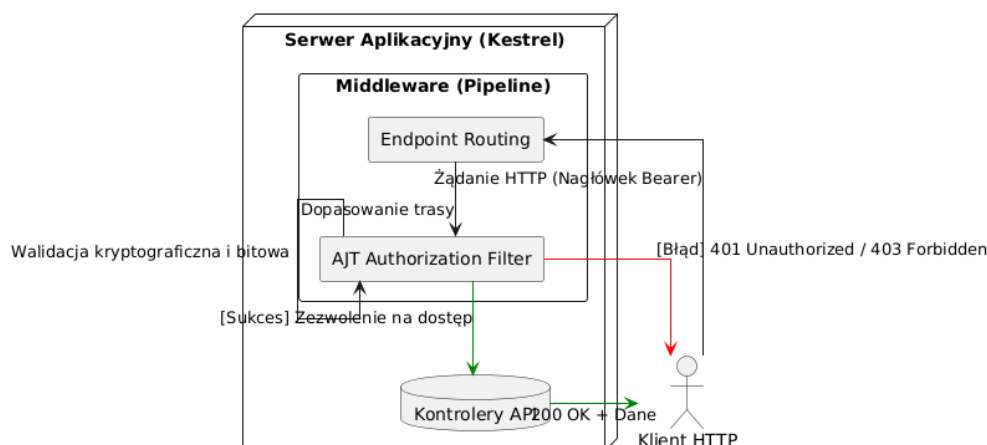
Rysunek 3.6. Diagram sekwencji weryfikacji uprawnień

4 Implementacja rozwiązania

Mając na uwadze przygotowany w poprzednim rozdziale teoretyczny model architektury, przystąpiono do właściwego programowania w języku C#. Zgodnie z wytycznymi czystego kodu (ang. *Clean Code*) oraz zasady rzemiosła oprogramowania (ang. *Software Craftsmanship*), w tej części pracy zaprezentowano wyłącznie wyselekcjonowane, kluczowe fragmenty kodu źródłowego. Zrezygnowano z przytaczania pełnych klas na rzecz wnikliwej analizy mechanizmów autoryzacyjnych.

4.1 Logika biznesowa i potok ASP.NET Core

Zanim użytkownik uzyska dostęp do chronionych zasobów, aplikacja musi przetworzyć jego zapytanie. Architektura platformy ASP.NET Core opiera się na potoku przetwarzania (ang. *Request Pipeline*), zbudowanym z oprogramowania pośredniczącego (*Middleware*) oraz filtrów. Nasz autorski system wstrzykuje swoją logikę decyzyjną dokładnie w ten potok, tuż przed przekazaniem sterowania do kontrolera biznesowego aplikacji, co zilustrowano na Rysunku 4.1.



Rysunek 4.1. Integracja filtra autoryzacyjnego z potokiem żądań ASP.NET Core

Zanim jednak filtr będzie mógł zwalidować żądanie (jak pokazano na schemacie), użytkownik musi najpierw wygenerować cyfrowe poświadczenie autentyczności, komunikując się z dedykowanym serwisem logowania.

4.2 Rejestracja i bezpieczne przechowywanie haseł

Fundamentem każdego systemu tożsamości jest bezpieczny proces rejestracji (ang. *Provisioning*). Serwis odpowiedzialny za ten proces upewnia się, że dane użytkownika są unikalne, a jego hasło nigdy nie trafia do bazy w postaci jawnej (tzw. *Plain text*).

W zaprezentowanej metodzie `Register` zastosowano mechanizm solenia i haszowania (`_passwordHasher.HashPassword`), co gwarantuje spełnienie wytycznych bezpieczeństwa OWASP. Metoda weryfikuje zajętość loginu oraz adresu e-mail, korzystając z wydzielonego zakresu kontenera zależności (DI).

```

public async Task<bool> Register(string username,
                                string email, string password)
{
    using var scope = _scopeFactory.CreateScope();
    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();

    var existingUserUsername = await userRepo.GetUserByLogin(username);
    var existingUserEmail = await userRepo.GetUserByLogin(email);

    if (existingUserUsername is not null || existingUserEmail is not null)
        return false;

    var user = new User
    {
        Username = username,
        Email = email,
        HashedPassword = _passwordHasher.HashPassword(password)
    };

    try
    {
        await userRepo.AddUser(user);
        return true;
    }
    catch
    {
        return false;
    }
}

```

4.3 Proces logowania i weryfikacji tożsamości

Punktem wejścia do bezpiecznej strefy systemu jest serwis `LoginService`. Odpowiada on za autentykację użytkownika na podstawie dostarczonych poświadczeń.

Poniższy fragment kodu prezentuje metodę `Login`. Warto zwrócić uwagę na wykorzystanie fabryki zakresów (`IServiceScopeFactory`). Ponieważ autoryzacja może być wywoływana z różnych kontekstów cyklu życia aplikacji, serwis ręcznie tworzy nowy zakres dla kontenera wstrzykiwania zależności (DI), z którego następnie bezpiecznie pobiera repozytorium użytkowników (`IUserRepo`). Po poprawnej weryfikacji skrótu hasła, proces generowania obiektów tokenowych delegowany jest do oddzielnej metody pomocniczej.

```

public async Task<CombinedToken?> Login(string login, string password)
{
    var hashedPassword = _passwordHasher.HashPassword(password);

    using var scope = _scopeFactory.CreateScope();
    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
    var user = await userRepo.GetUserByLogin(login);

    if (user is null || user.HashedPassword != hashedPassword)
        return null;

    return await CreateCombinedTokenForUser(scope, user);
}

```

Właściwy proces budowy, kompresji oraz kryptograficznego podpisywania poświadczeń został wyodrębniony do prywatnej metody pomocniczej `CreateCombinedTokenForUser`. Przyjmuje ona wcześniej wygenerowany zakres (ang. *Scope*) oraz zweryfikowany obiekt użytkownika.

To w tym miejscu wywoływany jest serwis odpowiedzialny za transformację tekstowych ról w skompresowaną maskę bitową (`EncodeRoles`). Następnie system generuje unikalny token odświeżający i zapisuje go w relacyjnej bazie danych. Zwieńczeniem procesu jest wywołanie metody `Hash`, która zamienia struktury obiektowe na docelowe ciągi znaków zabezpieczone algorytmem HMAC-SHA256.

```
private async Task<CombinedToken?> CreateCombinedTokenForUser(
    IServiceScope scope, User user)
{
    var userRoleRepo = scope.ServiceProvider.GetRequiredService<IUserRoleRepo>();
    var roles = await userRoleRepo.GetUserRoles(user);

    var token = new Token
    {
        Id = Guid.NewGuid(),
        UserId = user.Id,
        Data = await _tokenDataService.GetCustomTokenData(user.Id),
        ExpirationDate = DateTime.Now.Add(_options.Value.TokenExpirationTime)
    };

    if (roles is not null)
        token.UserRoles = await _roleService.EncodeRoles(
            roles.Select(x => x.RoleCode).ToList());

    var refreshTokenRepo = scope.ServiceProvider.GetRequiredService<IRefreshTokenRepo>();

    var refreshToken = new RefreshToken
    {
        UserId = user.Id,
        ExpirationDate = DateTime.Now.Add(_options.Value.RefreshTokenExpirationTime)
    };
    await refreshTokenRepo.AddRefreshToken(refreshToken);

    return new CombinedToken
    {
        Token = _hashingService.Hash(token),
        RefreshToken = _hashingService.Hash(refreshToken)
    };
}
```

4.4 Implementacja odświeżania sesji i rotacji tokenów

Zgodnie z wymaganiami architektonicznymi, czas życia wydanego tokenu (`TokenExpirationTime`) jest krótki. Proces podtrzymywania sesji znajduje się wewnątrz metody `Refresh`. Przyjmuje ona wygasający token odświeżający i implementuje proces rotacji kluczy w sposób bezpieczny transakcyjnie.

W pierwszej kolejności następuje weryfikacja kryptograficzna. Odrzuca ona zapytania ze zmodyfikowanym ładunkiem od razu, oszczędzając zasoby serwera bazy danych. Następnie system używa bloku `try-catch` do bezpiecznej deserializacji struktury. Kluczowym momentem algorytmu jest weryfikacja stanu tokenu w bazie danych i natychmiastowe usunięcie

(zużycie) starego poświadczenia przed wydaniem nowej pary, co zamyka wektor ataków typu *Replay Attack*.

```
public async Task<CombinedToken?> Refresh(string refreshTokenString)
{
    if (!_hashingService.VerifyHash(refreshTokenString))
        return null;

    RefreshToken refreshToken = null;
    try
    {
        var json = _hashingService.DecodePayload(refreshTokenString);
        refreshToken = JsonConvert.DeserializeObject<RefreshToken>(json);
        if (refreshToken is null) return null;
    }
    catch
    {
        return null;
    }

    using var scope = _scopeFactory.CreateScope();
    var refreshRepo = scope.ServiceProvider.GetRequiredService<IRefreshTokenRepo>();

    // Weryfikacja bazy danych i rotacja (usunięcie) zużytego tokenu
    var dbToken = await refreshRepo.GetRefreshToken(refreshToken.Id);
    if (dbToken is null || dbToken.ExpirationDate < DateTime.Now)
        return null;

    await refreshRepo.DeleteRefreshToken(dbToken.Id);

    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
    var user = await userRepo.GetUserById(dbToken.UserId);

    if (user is null)
        return null;

    return await CreateCombinedTokenForUser(scope, user);
}
```

4.5 Konfiguracja i dynamiczne rozszerzanie ładunku (Custom Payload)

Aby stworzona biblioteka mogła funkcjonować jako uniwersalne narzędzie typu *Plug and Play* w dowolnym projekcie, nie mogła ograniczać się wyłącznie do predefiniowanych pól. Zastosowano architektoniczny wzorzec strategii z wykorzystaniem delegatów `Func` zdefiniowanych w języku C#, wstrzykiwanych podczas konfiguracji aplikacji (wzorzec *Fluent API*).

Poniższy fragment pliku `Program.cs` prezentuje proces rejestracji biblioteki w głównym kontenerze zależności platformy ASP.NET Core. Za pomocą metody `AddDataToToken` deweloper wykorzystujący bibliotekę może przekazać własną funkcję (w tym przypadku `AddInfo`), która zostanie wywołana podczas każdego generowania tokenu.

```
builder.Services.UseAJT(
    new AJT.Options.AJTOptions
```

```

    {
        ConnectionString = builder.Configuration.GetConnectionString("AJT"),
        Secret = "super_sekretny_klucz",
        TokenExpirationTime = TimeSpan.FromMinutes(10),
        RefreshTokenExpirationTime = TimeSpan.FromDays(7),
        Roles = GenerateRoles()
    },
    config => config.UseRoleBootstrapper()
        .MigrateDatabase()
        .AddDataToToken(AddInfo)
);

static async Task<object> AddInfo(Guid userId, IServiceProvider services)
{
    using var scope = services.CreateScope();

    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
    var user = await userRepo.GetUserById(userId);

    return new { Department = "IT", AccessLevel = 1, Name = user.Username };
}

```

Od strony wewnętrznej biblioteki, obsługa tego delegata realizowana jest przez klasę `TokenDataService`. Przechwytuje ona zadeklarowaną funkcję i wywołuje ją asynchronicznie, przekazując identyfikator logującego się użytkownika oraz interfejs `IServiceProvider`. Dzięki dostępowi do dostawcy usług, funkcja zwrotna (*Callback*) może odpytywać dowolne inne moduły zarejestrowane w aplikacji nadrzędnej (np. system HR w celu pobrania struktury organizacyjnej).

```

internal sealed class TokenDataService : ITokenDataService
{
    private Func<Guid, IServiceProvider, Task<object>>? _func;
    private readonly IServiceProvider _serviceProvider;

    public static Func<Guid, IServiceProvider, Task<object>>? InitFunc { get; set; }

    public TokenDataService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
        _func = InitFunc;
    }

    public async Task<object?> GetCustomTokenData(Guid userId)
    {
        if (_func is null)
            return null;

        return await _func(userId, _serviceProvider);
    }
}

```

To właśnie metoda `GetCustomTokenData` jest wykonywana wewnątrz serwisu logowania podczas budowania obiektu `CombinedToken`. Zwrócone przez nią dane są automatycznie serializowane do formatu JSON. Jeżeli programista nie zdefiniuje własnej funkcji, metoda bezpiecznie zwróci wartość `null`, a struktura ładunku pozostanie zminimalizowana.

4.6 Automatyzacja zarządzania i migracja ról (Bootstrapper)

Mechanizm weryfikacji oparty na maskach bitowych jest wysoce zoptymalizowany, jednak z perspektywy inżynierii oprogramowania wymusza rygorystyczne zachowanie spójności między przypisanymi numerami bitów a konkretnymi kodami ról. Ręczne utrzymywanie takiej struktury byłoby podatne na błędy ludzkie (np. przypadkowa zamiana kolejności rejestrowania ról skutkowałaby trwałym przekłamaniem uprawnień w całym systemie).

Aby całkowicie zautomatyzować ten proces, zaimplementowano usługę działającą w tle – `RoleBootstrapper`. Klasa ta implementuje interfejs `IHostedService`, co sprawia, że jest uruchamiana przez platformę ASP.NET Core jeszcze przed przyjęciem pierwszego żądania HTTP. Algorytm w pierwszej kolejności pozyskuje listę wymaganych uprawnień (z opcji dostarczonych przez programistę lub poprzez skanowanie atrybutów w kodzie) i weryfikuje ich zbieżność ze stanem relacyjnej bazy danych.

```
internal class RoleBootstrapper : IHostedService
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly IOptions<AJTOptions> _options;

    public RoleBootstrapper(IServiceScopeFactory scopeFactory, IOptions<AJTOptions> options)
    {
        _scopeFactory = scopeFactory;
        _options = options;
    }

    public async Task StartAsync(CancellationToken cancellationToken)
    {
        using var scope = _scopeFactory.CreateScope();
        var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();

        var existingRoles = await roleRepo.GetAllRoles();
        var configRoles = new List<(int Id, string Code)>();

        var list = _options.Value.DetectRolesFromAssembly
            ? GetDefinedRoles()
            : _options.Value.Roles;

        if (list is null || list.Count == 0)
            return;

        for (int i = 0; i < list.Count; i++)
            configRoles.Add(new(i, list[i]));

        foreach (var (number, code) in configRoles)
        {
            var role = existingRoles.FirstOrDefault(x => x.RoleCode == code);

            if (role is null || role.RoleNumber != number)
            {
                await ChangeRolesAsync(configRoles, cancellationToken);
                return;
            }
        }
    }
}
```

```

    }

    public Task StopAsync(CancellationTokens cancellationTokens)
    {
        return Task.CompletedTask;
    }

```

4.6.1 Dynamiczne skanowanie atrybutów

Kluczowym elementem wspierającym ideę samokonfigurującej się biblioteki jest metoda `GetDefinedRoles`. Za pomocą narzędzi refleksji (ang. *Reflection*) z przestrzeni `System.Reflection`, algorytm przeszukuje aktualnie uruchomioną bibliotekę kodu (*Assembly*). Analizowane są zarówno definicje klas, jak i pojedynczych metod pod kątem występowania autorskiego atrybutu `AllowRoleAttribute`. Unikalny zbiór ról (*Distinct*) jest następnie zwracany do głównego algorytmu sprawdzającego.

```

private List<string> GetDefinedRoles()
{
    var assembly = Assembly.GetExecutingAssembly();

    var classRoles = assembly.GetTypes()
        .SelectMany(t => t.GetCustomAttributes<AllowRoleAttribute>(true))
        .SelectMany(a => a.Roles);

    var methodRoles = assembly.GetTypes()
        .SelectMany(t => t.GetMethods())
        .SelectMany(m => m.GetCustomAttributes<AllowRoleAttribute>(true))
        .SelectMany(a => a.Roles);

    return classRoles
        .Concat(methodRoles)
        .Distinct()
        .ToList();
}

```

4.6.2 Proces bezpiecznej migracji uprawnień

Jeśli `RoleBootstrapper` wykryje niespójność (nowa rola została dodana do kodu, lub zmieniła się ich kolejność), system wymusza całkowitą migrację danych bitowych. Operacja ta została zrealizowana wewnątrz prywatnej metody `ChangeRolesAsync`.

Proces przebiega w trzech fazach. W pierwszej fazie system tworzy w pamięci tymczasową kopię zapasową, łącząc identyfikatory użytkowników z ich tekstowymi kodami ról (aby uniknąć utraty powiązań na skutek zmiany numerów bitowych). Następnie następuje tzw. operacja czyszczenia środowiska, w której fizycznie kasowane są stare definicje ról i nadpisywane nowymi, uporządkowanymi maskami. W ostatniej fazie system iteruje przez zachowaną w pamięci kopię zapasową użytkowników, odszukuje nowo przypisane obiekty ról po ich tekstowych kodach i odtwarza relację wpisując je bezpiecznie do tabel asocjacyjnych.

```

private async Task ChangeRolesAsync(List<(int number, string Code)> roles,
    CancellationTokens cancellationTokens)
{
    using var scope = _scopeFactory.CreateScope();
    var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
    var userRoleRepo = scope.ServiceProvider.GetRequiredService<IUserRoleRepo>();
}

```

```

var currnetRoles = await roleRepo.GetAllRoles();
var userRoles = await userRoleRepo.GetAllUserRoles();
var userRoleNames = userRoles.Select(x => new
{
    x.UserId,
    currnetRoles.Find(role => role.Id == x.RoleId)?.RoleCode
}).ToList();

await roleRepo.RemoveAllRoles();

foreach (var (roleNumber, roleCode) in roles)
{
    await roleRepo.AddRole(new Entities.Role()
    {
        RoleNumber = roleNumber,
        RoleCode = roleCode
    });
}

var updatedRoles = await roleRepo.GetAllRoles();

foreach (var userRole in userRoleNames)
{
    var role = updatedRoles.FirstOrDefault(x => x.RoleCode == userRole.RoleCode);
    if (role is null)
        continue;

    await userRoleRepo.AddUserRole(
        new Entities.User() { Id = userRole.UserId }, role);
}
}
}

```

4.7 Implementacja operacji kryptograficznych i ochrona przed atakami

Zapewnienie integralności i niezaprzeczalności przesyłanych danych wymagało implementacji dedykowanego serwisu szyfrującego. Klasa `HashingService` realizuje pełen cykl życia tokenu: od kodowania, przez generowanie cyfrowej sygnatury, aż po weryfikację.

Aby wygenerowany ciąg znaków mógł być bezpiecznie przesyłany w nagłówkach HTTP (tzw. *Bearer Token*) bez ryzyka uszkodzenia formatu przesyłu, zaimplementowano niestandardowe kodowanie `Base64Url`. Zastępuje ono znaki kolidujące ze strukturą adresów URL i usuwa znaki dopełnienia (ang. *padding*). Rdzeniem kryptograficznym jest metoda `CreateSignature`, która wykorzystuje wbudowaną klasę `HMACSHA256` do połączenia ładunku z kluczem tajnym pobranym z bezpiecznej konfiguracji aplikacji.

```

private static string CreateSignature(string unsignedToken, string secret)
{
    var keyBytes = Encoding.UTF8.GetBytes(secret);
    var messageBytes = Encoding.UTF8.GetBytes(unsignedToken);

    using var hmac = new HMACSHA256(keyBytes);

```

```

        var hash = hmac.ComputeHash(messageBytes);

        return Base64UrlEncode(hash);
    }

    private static string Base64UrlEncode(byte[] input)
    {
        return Convert.ToBase64String(input)
            .TrimEnd('=')
            .Replace('+', '-')
            .Replace('/', '_');
    }

```

Proces generowania poświadczenia (metoda `Hash`) polega na serializacji struktury obiektowej języka C# do formatu JSON, zakodowaniu jej wyżej opisanym mechanizmem, wyliczeniu skrótu matematycznego, a następnie połączeniu obu tych elementów znakiem kropki. Wynikiem jest gotowy do wysłania, skompresowany autorski token. Do ekstrakcji danych z tokenów służy metoda odwrotna `DecodePayload`.

```

    public string Hash(Token token)
    {
        var tokenJson = JsonConvert.SerializeObject(token);
        var tokenJson64 = Base64UrlEncode(Encoding.UTF8.GetBytes(tokenJson));
        var signature = CreateSignature(tokenJson64, _options.Value.Secret);

        return string.Join('.', tokenJson64, signature);
    }

    public string DecodePayload(string hashedToken)
    {
        var parts = hashedToken.Split('.');
        if (parts.Length != 2)
            throw new ArgumentException("Invalid AJT");

        var payload = parts[0];
        var bytes = Base64UrlDecode(payload);

        return Encoding.UTF8.GetString(bytes);
    }

```

Najbardziej newralgicznym punktem z perspektywy wektorów ataków sieciowych jest weryfikacja nadesłanego podpisu. Metoda `VerifiyHash` rozdziela przysłany ciąg znaków, a następnie samodzielnie generuje nową sygnaturę na podstawie przechwyconego ładunku i własnego klucza tajnego.

Podczas weryfikacji tożsamości zidentyfikowano potencjalne zagrożenie związane z atakami czasowymi (ang. *Timing Attacks*). Standardowy operator porównania ciągów znaków języka C# przerywa działanie natychmiast po napotkaniu pierwszej różnicy, co pozwalałoby atakującemu na iteracyjne odgadnięcie całego skrótu na podstawie analizy mikrosekundowych opóźnień odpowiedzi serwera. Aby temu zapobiec, zaimplementowano stałoczasową weryfikację `FixedTimeEquals`, maskującą rzeczywisty czas trwania ewaluacji ciągów.

```

    public bool VerifiyHash(string token)
    {
        var parts = token.Split('.');
        if (parts.Length != 2)

```

```

        return false;

        string unsignedToken = parts[0];
        string signature = parts[1];
        string expectedSignature = CreateSignature(unsignedToken, _options.Value.Secret);

        var signatureBytes = Encoding.UTF8.GetBytes(signature);
        var expectedBytes = Encoding.UTF8.GetBytes(expectedSignature);

        return CryptographicOperations.FixedTimeEquals(signatureBytes, expectedBytes);
    }

```

4.8 Filtry potoku HTTP i weryfikacja tożsamości

Zwieńczeniem całego systemu jest warstwa przechwytyjąca żądania sieciowe. Architektura platformy ASP.NET Core pozwala na wpięcie autorskiej logiki decyzyjnej za pomocą interfejsu `IAsyncAuthorizationFilter`. Aby zachować zgodność z dobrymi praktykami inżynierii oprogramowania (wzorzec *Single Responsibility Principle*), mechanizm walidacji podzielono na dwa niezależne filtry.

Pierwszy z nich, `RequireAuthenticationFilter`, służy wyłącznie do procesu uwierzytelniania. Jego zadaniem jest wyodrębnienie nagłówka `Bearer`, weryfikacja sumy kontrolnej za pomocą serwisu kryptograficznego oraz upewnienie się, że czas życia tokenu nie minął. Niezmiernie istotnym elementem tego filtru jest ekstrakcja niestandardowego ładunku (ang. *Custom Payload*) i wstrzyknięcie go do słownika `HttpContext.Items`. Dzięki temu operacji tej nie trzeba powtarzać, a wywoływane w dalszej kolejności kontrolery biznesowe zyskują natychmiastowy dostęp do dodatkowych danych o użytkowniku. Każde niepowodzenie na tym etapie skutkuje przerwaniem potoku i zwróceniem kodu 401 `Unauthorized`.

```

internal sealed class RequireAuthenticationFilter : IAsyncAuthorizationFilter
{
    private readonly IHashingService _hashingService;

    public RequireAuthenticationFilter(IHashingService hashingService)
    {
        _hashingService = hashingService;
    }

    public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
    {
        var authHeader = context.HttpContext.Request.Headers["Authorization"].ToString();

        if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
        {
            context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
            await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
            return;
        }

        var token = authHeader["Bearer ".Length..].Trim();

        if (!_hashingService.VerifyHash(token))
        {
            context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
            await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
            return;
        }
    }
}

```

```

        return;
    }

    try
    {
        var payloadJson = _hashingService.DecodePayload(token);
        var tokenObject = JsonConvert.DeserializeObject<Token>(payloadJson);
        if (tokenObject is null || string.IsNullOrEmpty(tokenObject.UserRoles))
            throw new UnauthorizedException();

        var data = tokenObject.Data;
        if (data is not null)
            context.HttpContext.Items["AJT"] = data;

        if (tokenObject.ExpirationDate < DateTime.Now)
        {
            throw new UnauthorizedException();
        }
    }
    catch
    {
        context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject());
        return;
    }

    await Task.CompletedTask;
}
}

```

4.9 Zaawansowana autoryzacja oparta o role

Drugi filtr, `AllowRoleFilter`, stanowi rozszerzenie mechanizmu podstawowego. Oprócz pełnej ścieżki weryfikacji integralności i ważności tokenu, jego głównym zadaniem jest autoryzacja dostępu do konkretnego zasobu.

Po pomyślnym zdekodowaniu poświadczenia, filtr odczytuje szesnastkową maskę ról i wykorzystuje wstrzyknięty serwis `IRoleService` do jej transformacji na zbiór przyznanych uprawnień. Następnie realizowana jest operacja przecięcia zbiorów (ang. *Intersection*) – system upewnia się, czy użytkownik posiada przynajmniej jedną z ról wymaganych przez punkt końcowy. Ścisłe rozdzielanie błędów logicznych, obsługiwane blokiem `try-catch`, gwarantuje, że brak wymaganej roli zwraca poprawny semantycznie kod 403 `Forbidden`, podczas gdy wszelkie manipulacje kryptograficzne skutkują błędem 401 `Unauthorized`.

```

internal sealed class AllowRoleFilter : Attribute, IAsyncAuthorizationFilter
{
    private readonly string[] _roles;
    private readonly IHashingService _hashingService;
    private readonly IRoleService _roleService;

    public AllowRoleFilter(string[] roles, IHashingService hashingService, IRoleService roleService)
    {
        _roles = roles;
        _hashingService = hashingService;
        _roleService = roleService;
    }
}

```

```
}
```

```
public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
{
    var authHeader = context.HttpContext.Request.Headers["Authorization"].ToString();

    if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("Bearer "))
    {
        context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
        return;
    }

    var token = authHeader["Bearer ".Length..].Trim();
    if (!_hashingService.VerifyHash(token))
    {
        context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
        return;
    }

    try
    {
        var payloadJson = _hashingService.DecodePayload(token);
        var tokenObject = JsonConvert.DeserializeObject<Token>(payloadJson);
        if (tokenObject is null || string.IsNullOrEmpty(tokenObject.UserRoles))
            throw new UnauthorizedException();

        var data = tokenObject.Data;
        if (data is not null)
            context.HttpContext.Items["AJT"] = data;

        if (tokenObject.ExpirationDate < DateTime.Now)
        {
            throw new UnauthorizedException();
        }

        var userRoles = await _roleService.DecodeRoles(tokenObject.UserRoles);
        foreach (var role in _roles)
            if (userRoles.Contains(role))
                return;

        throw new ForbidException();
    }
    catch (ForbidException)
    {
        context.HttpContext.Response.StatusCode = StatusCodes.Status403Forbidden;
        await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
        return;
    }
    catch (Exception)
    {
        context.HttpContext.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.HttpContext.Response.WriteAsync(JsonConvert.SerializeObject(new { }));
        return;
    }
}
```

}
}
}

5 Analiza bezpieczeństwa i testowanie

Istotnym etapem cyklu życia oprogramowania nastawionego na autoryzację jest empiryczna walidacja jego podatności. Rozdział ten skupia się na ocenie zaprojektowanego systemu przez pryzmat uznanego w branży standardu OWASP Top 10, szczegółowym omówieniu zautomatyzowanych testów bezpieczeństwa, teoretycznej analizie wydajności oraz nakreśleniu perspektyw dalszego rozwoju w aspektach, które wykraczały poza ramy obecnej iteracji prac.

5.1 Zgodność ze standardem OWASP Top 10

OWASP Top 10 to najpopularniejszy, globalny dokument uświadamiający zagrożenia bezpieczeństwa aplikacji internetowych. Biblioteka autoryzacyjna została zaprojektowana w oparciu o ideę *Security by Design*, co ogranicza ekspozycję na najczęstsze z klasyfikowanych podatności:

- **Broken Access Control (Naruszenie kontroli dostępu):** W modelu Zero Trust każda metoda API pozbawiona weryfikacji stanowi ryzyko eskalacji uprawnień. System eliminuje to zagrożenie poprzez sztywne wpięcie logiki decyzyjnej do potoku wykonawczego. Co więcej, rygorystyczne operacje oparte na matematycznych maskach bitowych nie pozwalają na manipulację ciągami tekstowymi ról (np. próbę nadpisania parametru `role="admin"`) przez nieuprzywilejowanych użytkowników.
- **Cryptographic Failures (Błędy kryptograficzne):** Poufność haseł została zapewniona poprzez wykorzystanie silnych skrótów kryptograficznych wraz z mechanizmem solenia (ang. *Salting*), co całkowicie uniemożliwia odzyskanie haseł tekstowych nawet w przypadku fizycznego wycieku bazy danych. Integralność samego tokenu opiera się o sprawdzony standard z rodziny SHA-256 i unikalny sekretny klucz.
- **Injection (Wstrzykiwanie kodu):** System wykorzystuje mapowanie obiektowo-relacyjne Entity Framework Core, które pod warstwą abstrakcji zawsze używa zapytań parametryzowanych. Architektura ta całkowicie neutralizuje zagrożenia związane z SQL Injection, uniemożliwiając wykonanie preparowanych zapytań modyfikujących strukturę relacyjną.
- **Identification and Authentication Failures (Błędy identyfikacji):** Implementacja rotacji tokenów odświeżających (opisana w poprzednich rozdziałach) oraz natychmiastowe usuwanie zużytych poświadczeń skutecznie chroni aplikację przed atakami typu *Session Fixation* oraz *Replay Attack* (atak polegający na przechwyceniu i ponownym wykorzystaniu starego żądania uwierzytelniającego).
- **Security Misconfiguration (Błędy konfiguracji zabezpieczeń):** Biblioteka wymusza na programiście dostarczenie klucza szyfrującego z poziomu konfiguracji zewnętrznej (poprzez wstrzyknięcie obiektu `AJTOptions`). Wymusza to odseparowanie kluczy od kodu źródłowego, chroniąc przed ich przypadkowym ujawnieniem w repozytoriach publicznych (np. GitHub).
- **Security Logging and Monitoring Failures (Brak logowania i monitorowania):** W architekturze opartej na wstrzykiwaniu zależności (DI) filtry autoryzacyjne mogą z łatwością przyjmować natywny interfejs `ILogger`. Pozwala to na automatyczne rejestrowanie każdego odrzuconego żądania (kod 401 i 403), co jest kluczowe z punktu widzenia systemów SIEM (ang. *Security Information and Event Management*) wykrywających anomalie w ruchu sieciowym.

5.2 Automatyzacja testów bezpieczeństwa kryptograficznego

Aby zagwarantować najwyższy poziom niezawodności i odporności systemu na najczęstsze wektory ataków, zaimplementowano zestaw automatycznych testów jednostkowych. Wykorzystano w tym celu środowisko `xUnit` oraz bibliotekę `FluentAssertions`, która pozwala na tworzenie czytelnych i deklaratywnych asercji. Głównym obiektem testowanym (ang. *System Under Test*) jest klasa `HashingService`.

Poniższy kod prezentuje trzy kluczowe scenariusze testowe, weryfikujące poprawność mechanizmu walidacji tokenów:

W pierwszym teście symulowany jest atak polegający na modyfikacji uprawnień (tzw. *Privilege Escalation*). Atakujący przechwytuje własny, legalnie wydany token, rozdziela go, a następnie dekoduje ładunek (ang. *Payload*). Wewnątrz czystego formatu JSON zmienia swoją maskę ról z niższej na wyższą (00 na 01), po czym ponownie koduje ciąg za pomocą algorytmu `Base64Url`. Zmodyfikowany ładunek łączy ze starą, oryginalną sygnaturą i odsyła do serwera. Asercja weryfikuje, czy system poprawnie odrzuci tak sfabrykowany token z powodu niezgodności sumy kontrolnej.

Drugi scenariusz testuje odporność na ataki polegające na podrabianiu tokenów przez zewnętrzne serwery (np. aplikacje stron trzecich). Test generuje całkowicie poprawny strukturalnie token, używając jednak do jego podpisania obcego klucza tajnego. Weryfikacja udowadnia, że bez znajomości wewnętrznego klucza serwera głównego, sfałszowanie podpisu jest niemożliwe.

Ostatni blok testowy, wykorzystujący atrybut `[Theory]`, to testy odpornościowe (ang. *Fuzz testing*). Do serwisu wstrzykiwane są uszkodzone, niepełne lub puste ciągi znaków. Asercje potwierdzają, że algorytm bezpiecznie obsługuje błędy formatowania, zwracając wartość logiczną `false`, zamiast generować krytyczne błędy wykonania (ang. *Exceptions*), które mogłyby doprowadzić do ataku typu *Denial of Service*.

```
using AJT.Models;
using AJT.Options;
using AJT.Services;
using FluentAssertions;

namespace AJT.Tests
{
    public class HashingServiceTests
    {
        private readonly HashingService _sut;

        public HashingServiceTests()
        {
            var options = Microsoft.Extensions.Options.Options.Create(
                new AJTOptions { Secret = "BardzoTajnyKluczSerwera1234567890" });
            _sut = new HashingService(options);
        }

        [Fact]
        public void VerifyHash_GivenTamperedToken_ShouldReturnFalse()
        {
            var token = new Token { Id = Guid.NewGuid(), UserRoles = "00" };
            var hashedToken = _sut.Hash(token);

            var parts = hashedToken.Split('.');
            var originalSignature = parts[1];
```

```

        var jsonPayload = _sut.DecodePayload(hashedException);

        var tamperedJson = jsonPayload.Replace("\"00\"", "\"01\"");

        var tamperedBytes = System.Text.Encoding.UTF8.GetBytes(tamperedJson);
        var tamperedPayload64 = Convert.ToBase64String(tamperedBytes)
            .TrimEnd('=')
            .Replace('+', '-')
            .Replace('/', '_');

        var tamperedToken = $"{tamperedPayload64}.{originalSignature}";

        var isValid = _sut.VerifyHash(tamperedToken);

        isValid.Should().BeFalse();
    }

    [Fact]
    public void VerifyHash_SignedWithDifferentKey_ShouldReturnFalse()
    {
        var attackerOptions = Microsoft.Extensions.Options.Options.Create(
            new AJTOptions { Secret = "ZupelnieInnyKluczAtakujacego0987654321" });
        var attackerSut = new HashingService(attackerOptions);

        var forgedToken = attackerSut.Hash(new Token { Id = Guid.NewGuid() });

        var isValid = _sut.VerifyHash(forgedToken);

        isValid.Should().BeFalse();
    }

    [Theory]
    [InlineData("TylkoJedenSegmentBezKropki")]
    [InlineData("Segment1.Segment2.Segment3.ZaDuzoKropek")]
    [InlineData(".TylkoPodpis")]
    [InlineData("TylkoLadunek.")]
    [InlineData("")]
    [InlineData(" ")]
    public void VerifyHash_GivenMalformedFormat_ShouldReturnFalse(string malformedInput)
    {
        var isValid = _sut.VerifyHash(malformedInput);

        isValid.Should().BeFalse();
    }
}

```

5.3 Analiza wydajnościowa i narzut obliczeniowy

Główną motywacją do stworzenia autorskiego systemu opartego na maskach bitowych była chęć redukcji narzutu wydajnościowego, jaki generują standardowe tokeny JWT. Należy podkreślić, że uwierzytelnianie i autoryzacja odbywają się przy każdym pojedynczym żądaniu HTTP skierowanym do chronionego punktu końcowego, co czyni ten mechanizm jednym z

najbardziej obciążonych elementów aplikacji.

Odrzucenie klasycznego modelu, w którym uprawnienia zapisywane są jako obszerne tablice ciągów znaków (np. ["Administrator", "Moderator", "Uzytkownik"]), na rzecz skompresowanego formatu szesnastkowego, diametralnie zmniejsza rozmiar pojedynczego żądania (ang. *Payload Size*). Przekłada się to bezpośrednio na redukcję przepustowości sieciowej oraz zmniejszenie alokacji pamięci operacyjnej podczas deserializacji JSON.

Kolejnym aspektem jest wydajność procesora (CPU). Samo generowanie skrótów kryptograficznych przy pomocy symetrycznego algorytmu HMAC-SHA256 narzuca bardzo małe opóźnienia obliczeniowe w porównaniu z weryfikacją asymetryczną (np. wykorzystującą klucze RSA). Dzięki użyciu operatorów bitowych i wydajnych zbiorów (*HashSet*) podczas operacji przeszukiwania ról wewnątrz filtra HTTP, ewaluacja praw dostępu wykonywana jest w czasie stałym $O(1)$, co gwarantuje wysoką skalowalność nawet przy ogromnym obciążeniu systemu.

5.4 Ograniczenia systemu i obszary dalszego rozwoju

Świadomość ograniczeń wyprodukowanego kodu stanowi miarę inżynierskiej dojrzałości projektu. Choć system spełnia założenia optymalizacyjne w zakresie objętości danych i szybkości działania, jego implementacja pozostawia pole do rozbudowy w przyszłych iteracjach badawczych.

Czarna lista tokenów (Blacklisting): W pierwszej kolejności nie zaimplementowano tzw. czarnej listy tokenów. Ponieważ serwer z założenia nie odpytuje bazy danych przy walidacji głównego tokenu dostępowego, w przypadku jawnego wylogowania token pozostaje ważny aż do czasu naturalnego wygaśnięcia. Wdrożenie zewnętrznej, błyskawicznej bazy pamięciowej (np. Redis), która przechowywałaby identyfikatory unieważnionych tokenów z przypisanym czasem życia dopasowanym do ich wygaśnięcia, skutecznie załatałoby tę lukę bez drastycznego spadku wydajności.

Kryptografia asymetryczna: Brak implementacji kryptografii asymetrycznej ogranicza wdrożenie systemu do prostszych środowisk. Obecny symetryczny algorytm sprawdza się w przypadku monolitycznego API, lecz w wysoce rozproszonej architekturze wymagane byłoby współdzielenie tego samego klucza tajnego między wszystkimi mikroserwisami. Wprowadzenie wsparcia dla algorytmów z rodziny RSA (gdzie główny serwer autoryzacyjny używa klucza prywatnego do podpisu, a mikroserwisy używają klucza publicznego wyłącznie do weryfikacji) stanowiłoby krytyczne rozszerzenie biblioteki.

Ochrona przed atakami Brute-Force: Autorski system nie posiada wbudowanych mechanizmów limitowania zapytań sieciowych. Konsekwencją tego jest teoretyczna podatność punktu końcowego realizującego funkcję logowania na ataki siłowe lub ataki słownikowe (ang. *Credential Stuffing*). W przyszłości biblioteka powinna zostać zintegrowana z natywnym modulem *ASP.NET Core Rate Limiting*, aby automatycznie blokować numery IP generujące anomalną liczbę błędnych prób uwierzytelnienia.

Wsparcie dla Multi-Factor Authentication (MFA): Obecny przepływ uwierzytelniania opiera się wyłącznie na jednym składniku (wiedzy – hasle użytkownika). Dodanie wsparcia dla uwierzytelniania wieloskładnikowego, np. z wykorzystaniem algorytmu TOTP (ang. *Time-based One-Time Password*) lub autoryzatorów zewnętrznych, znacząco podniosłoby ocenę wiarygodności tożsamości w modelu Zero Trust.

Automatyczna rotacja kluczy kryptograficznych: W obecnej wersji klucz szyfrujący podawany w obiekcie opcji jest statyczny. Dobrą praktyką w rozbudowanych systemach jest implementacja tzw. *Key Rollover*, gdzie klucze tajne rotują automatycznie co określoną liczbę dni, a stare sygnatury pozostają wspierane jedynie przez okres przejściowy. Wdrożenie tej funkcji wyeliminowałoby ryzyko związane ze skompromitowaniem długowiecznego klucza aplikacji.

6 Podsumowanie

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie autorskiego, zoptymalizowanego systemu uwierzytelniania i autoryzacji dla aplikacji opartych o środowisko ASP.NET Core. Poprzez analizę powszechnie stosowanych technologii sieciowych oraz wymagań współczesnych norm bezpieczeństwa, udowodniono skuteczność paradygmatu *Zero Trust* wspomaganego szybkimi, bezstanowymi tokenami.

Opracowana biblioteka pomyślnie rozwiązuje problem strukturalnego narzutu przesyłanych danych obecnego w popularnym standardzie JWT. Eliminacja nieużywanych metadanych oraz innowacyjne, oparte na konwersji bitowej podejście do interpretowania złożonych struktur ról użytkowników zaowocowały miniaturyzacją autoryzacyjnych żądań sieciowych. Wykorzystanie warstwy abstrakcji programistycznej, zgodnej z wzorcami wstrzykiwania zależności, umożliwiło zamknięcie całej złożoności w zgrabnym, hermetycznym module o minimalnym progu wejścia (ang. *low entry barrier*) dla przyszłych programistów.

Przeprowadzona analiza testowa w oparciu o globalne wytyczne OWASP potwierdziła, że kompresja algorytmów nie wpłynęła na obniżenie parametrów zabezpieczeń kryptograficznych. Choć uwypuklono obszary do potencjalnej kontynuacji badań technologicznych – takie jak implementacja natywnych rozwiązań pamięci podręcznej dla autoryzacji czy kluczy asymetrycznych – wykreowane rozwiązanie spełnia zadane wymagania inżynierskie i stanowi w pełni funkcjonalną alternatywę autoryzacyjną dla systemów o klasycznym i mikroserwisowym profilu architektonicznym.

Wyrażam zgodę na udostępnienie mojej pracy w czytelniach Biblioteki SGGW w tym w Archiwum Prac Dyplomowych SGGW.

.....
(czytelny podpis autora pracy)

